

Claude Code + MCP Domain 2 — Mixed Review

Claude Code (20%) + MCP Integration
(18%)

38% of CCA-F exam

10 questions · Mixed
topics

Session result

9 / 10

Outstanding performance — strongest quiz yet!

Week 2 average: 18/25 = 72% (pass threshold: 72%)

Question scorecard

Q	Topic	Ans	Result
Q1	CLAUDE.md precedence — no local file	B	✓ Correct
Q2	Slash command precedence — project vs global	B	✓ Correct
Q3	CI prompt — missing exit code + JSON output	B	✓ Correct
Q4	Overlapping tools — reasoning overload	B	✓ Correct
Q5	Adding required field — breaking change	B	✓ Correct
Q6	Monolithic tool — no human confirmation	B	✓ Correct
Q7	Schema validation error — self-correct & retry	C	✓ Correct
Q8	Local vs remote server architecture	D	✓ Correct
Q9	Minimal footprint + tool boundary violation	B	✓ Correct
Q10	Slash command — vague + missing argument usage	C	✗ Missed (correct: D)

Key takeaway from missed question

Q10 — Slash command argument handling (your answer: C, correct: D)

- There is NO \$1, {arg}, or formal parameter syntax in slash command files.
- Arguments are natural language context — described in prose.
- Correct: 'Review the file provided as the argument for security vulnerabilities.'
- Q10 had TWO problems: vague description AND missing argument usage instruction.
- D captures BOTH. C only identified \$1 (which doesn't exist).

Full question bank with explanations

All 10 questions with full explanations. Q10 highlighted in red.

Q1. Global CLAUDE.md: '2-space indentation.' Project: '4-space indentation.' No local CLAUDE.md. Which rule applies in /project/src/?

- A) 2-space — global always takes priority
- ✓ B) 4-space — project overrides global when no local file
- C) Both apply — Claude averages to 3-space
- D) Neither — conflicting rules cancel

Explanation:

- B correct: No local CLAUDE.md → project is most specific. Project overrides global. Local > Project > Global.
- A wrong: Global is LOWEST priority. C and D wrong: Most specific level wins.

Q2. Files: ~/.claude/commands/format.md, .claude/commands/format.md, .claude/commands/deploy.md. Developer runs /format. Which executes?

- A) ~/.claude/commands/format.md — global takes precedence
- ✓ B) .claude/commands/format.md — project overrides global
- C) Both run and outputs merge
- D) Error — duplicate names not allowed

Explanation:

- B correct: Slash commands follow same specificity as CLAUDE.md. Project overrides global.
- A wrong: Global = lowest priority. C wrong: One executes. D wrong: Resolved by precedence.

Q3. CI: `claude --print 'Analyse codebase for unused dependencies. List them and suggest removal.'` Pipeline passes even when 15 deps found. What is missing?

A) The `--ci` flag

✓ **B) JSON output format + explicit exit code instruction**

C) Reference to `CLAUDE.md` — CI doesn't load automatically

D) The `--headless` flag

Explanation:

- B correct: No JSON format (pipeline gets prose it can't parse) + No exit code instruction (Claude always exits 0).
- A wrong: No `--ci` flag. `--print` already used. C wrong: `CLAUDE.md` loads automatically. D wrong: No `--headless` flag.

Q4. MCP server exposes: `get_record`, `fetch_record`, `retrieve_record`, `lookup_record` — all query a DB record by ID. Primary design problem?

A) Too many tools — merge into one `database_tool`

✓ **B) Overlapping tools cause reasoning overload — merge into one `query_record`**

C) Names too similar — rename

D) Four tools violates MCP limit

Explanation:

- B correct: Four tools doing the same thing = overlapping violation. Claude can't reliably choose.
- A wrong: `database_tool` = over-broad violation. C wrong: Renaming doesn't fix overlap. D wrong: No MCP tool limit.

Q5. MCP tool requires `user_id` + `timestamp`. Team adds new required field `request_source`. Type of change?

A) Backward-compatible — modify in-place

✓ **B) Breaking — existing calls without `request_source` fail validation. Version the tool.**

C) Backward-compatible — new fields optional by default

D) Breaking — only if existing prompts don't pass it

Explanation:

- B correct: Adding a `REQUIRED` field = more restrictive = breaking. Every existing call without it fails.
- A wrong: Not backward-compatible. C wrong: Fields are not optional by default. D wrong: Breaking is universal.

Q6. Colleague proposes process_order tool: validates + charges card + updates inventory + sends email + logs audit in one call. Strongest argument against?

A) Tool name too vague

✓ **B) Impossible to pause for human confirmation before irreversible steps like charging a card**

C) Exceeds MCP execution time limit

D) Monolithic tools cannot be registered

Explanation:

- B correct: Monolithic tools eliminate human confirmation checkpoints. Charging a card is irreversible.
- A wrong: Renaming doesn't fix architecture. C wrong: No MCP execution limit. D wrong: Monolithic tools can be registered.

Q7. MCP tool returns: {error: 'VALIDATION_ERROR', message: 'Required field user_id is missing', retryable: true}. What should the agent do?

A) Retry with exponential backoff — error is retryable

B) Treat as permanent failure, escalate

✓ **C) Self-correct by adding missing user_id, then retry once**

D) Wait 30s and retry exact same request

Explanation:

- C correct: Schema validation error — malformed input. Fix the input first, then retry. Retrying same request always fails.
- A wrong: Backoff is for transient errors. B wrong: Escalating is disproportionate. D wrong: Same input = same error.

Q8. Team needs: local developer file access, shared cloud knowledge base queries, Slack notifications. How many servers and types?

A) One remote HTTP for all three

B) One local stdio for all three

C) Three — one local stdio, two remote HTTP

✓ **D) Two — one local stdio for files, one remote HTTP for KB + Slack combined**

Explanation:

- D correct: Local files = local stdio (needs direct machine access). KB + Slack = shared network resources → one remote HTTP handles both.
- A wrong: Remote can't access local file systems. B wrong: Local stdio can't serve shared cloud resources. C wrong: Over-engineered.

Q9. search_customers MCP tool has insert/update/delete credentials though it only searches. Which two principles violated?

- A) Tool granularity and schema validation
- ✓ **B) Minimal footprint and tool boundary design**
- C) Namespacing and tool composition
- D) Server versioning and error handling

Explanation:

- B correct: Minimal footprint — far more permissions than needed. Tool boundary design — search tool with write capabilities violates single responsibility.
- A wrong: Schema validation is about input types. C and D wrong: Unrelated to permission scoping.

Q10. Team adds /review slash command. File: '# /review\nCheck the file for issues.'
What are the TWO problems?

Your answer: C

- A) Uses markdown instead of JSON
- B) Description too vague and no output format
- C) Missing formal argument parameter like \$1
- ✓ **D) Both B and the missing instruction on how to use the filename argument**

Explanation:

- D correct: TWO problems — vague description ('Check the file for issues') AND no instruction on how to use the filename argument.
- C wrong (the trap): There is NO \$1 syntax. Slash command files are markdown natural language — arguments described in prose.
- B alone insufficient — misses the argument usage instruction problem.
- Exam rule: Slash command arguments = natural language context, never formal parameters.

Week 2 complete — full summary

Day	Topic	Score	Review area
Day 6	CLAUDE.md hierarchy & Claude Code commands	2/3	CI flag (--print) confusion
Day 7	Slash commands & CI/CD workflows	2/3	Multiple problems in one CI scenario
Day 8	MCP fundamentals & tool design	3/3 ■	—
Day 9	MCP integration patterns & boundaries	2/3	Permissiveness rule (Required→Optional = safe)
Day 10	Domain 2 mixed review	9/10 ■	Slash command argument syntax (\$1 does not exist)

**Week 3 starts
Monday**

Prompt engineering & structured output — JSON schemas, few-shot techniques, validation retry loops. Domain weight: 20%.